PREUVE DE CONCEPT · CITYVISION

Segmenter la ville avec EoMT

Un modèle récent face à la baseline ResNet50-UNet, sur Cityscapes à 9 classes

Darya Filatova · présentation à Karim · octobre 2026

Point de départ : 0,816 de mIoU

CityVision segmente des photos de rue en **9 classes** : route, bâtiment, végétation, ciel, personne, voiture, panneau, vélo et fond.

Le modèle en production est un **ResNet50-UNet** : chaque pixel reçoit une classe. Il est servi par l'API et l'application du projet.

Question de la preuve de concept : un modèle récent fait-il mieux, dans les mêmes conditions ?

BASELINE ACTUELLE

0,816

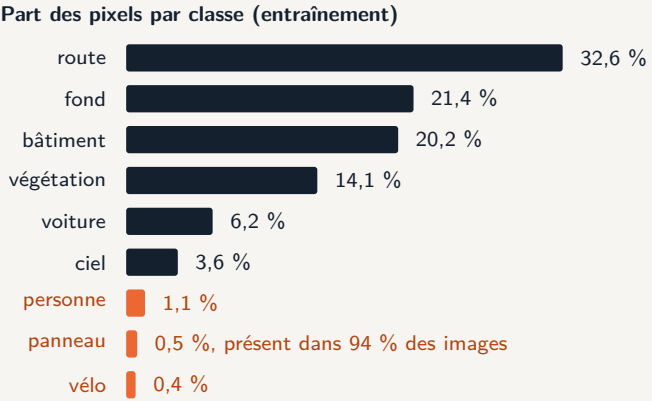
mIoU sur les 500 images de validation (moyenne des IoU des 9 classes)

Cityscapes, ramené à 9 classes

2 975
images d'entraînement, 18 villes

500
images de validation, 3 villes

512×1024
résolution utilisée (originale 2048×1024)



Modèle retenu : EoMT (CVPR 2025)

Étape intermédiaire : Mask2Former (CVPR 2022). EoMT date de mars 2025 : moins de 5 ans.

Changement de paradigme

Le modèle prédit un ensemble de masques, chacun avec une classe, au lieu d'une classe par pixel.

Petits objets

Un masque couvre un objet entier : gain attendu sur personnes, panneaux et vélos, les classes faibles de la baseline.

Simple et rapide

Un Vision Transformer seul, sans décodeur : précision comparable à l'état de l'art, jusqu'à 4 × plus rapide selon les auteurs.

Faisable

Code officiel sous licence MIT, intégré à Hugging Face Transformers, poids pré-entraînés disponibles.

Trois références de recherche

Algorithme

Your ViT is Secretly an Image Segmentation Model
Kerssies et al., 2025 · arXiv:2503.19108 · EoMT, mis en œuvre ici

Fondation

Masked-attention Mask Transformer for Universal Image Segmentation
Cheng et al., CVPR 2022 · arXiv:2112.01527 · Mask2Former, base d'EoMT

Données

The Cityscapes Dataset for Semantic Urban Scene Understanding
Cordts et al., CVPR 2016 · arXiv:1604.01685 · jeu de données et protocole

Code réutilisé et cité : Hugging Face Transformers et la recette d'entraînement du dépôt EoMT (licence MIT). Données, pipeline, évaluation et dashboard sont propres au projet.

Une comparaison équitable

Identique pour les 3 modèles

- Mêmes images, 9 classes, 512×1024
- Même échantillonneur pondéré (classes rares)
- Même budget : 20 époques, lot effectif de 4, arrêt anticipé
- Même métrique, même code d'évaluation
- Aucun point de départ n'a vu Cityscapes

Propre à chaque modèle

- Pré-entraînement : ImageNet, ADE20K, COCO
- Recette de son article : perte, optimiseur, calendrier du taux d'apprentissage
- Lots de 2 et accumulation de gradient pour les transformers (8 Go de GPU)
- Précision bf16 pour les transformers

Mesures : mIoU et IoU par classe, temps d'inférence, bootstrap apparié sur les 500 images.

Des pixels à un ensemble de masques

Baseline : une décision par pixel

$$\hat{y}(x) = \arg\max_{c \in \mathcal{C}} \; z_c(x)$$

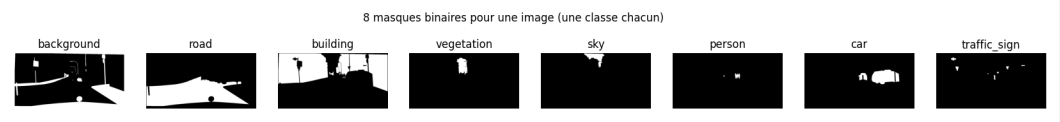
524 288 classifications indépendantes par image.

Mask2Former, EoMT : N paires (classe, masque)

$$\left\{(\mathbf{p}_q,\, m_q)\right\}_{q=1}^{N}, \qquad N \in \{100,\, 200\}$$

$$S_c(x) = \sum_{q=1}^{N} p_q(c)\, \sigma\big(m_q(x)\big), \qquad \hat{y}(x) = \arg\max_{c} S_c(x)$$

Côté données : $\mathcal{G}(y) = \left\{(c_j,\, g_j)\right\}_{j=1}^{K}, \qquad g_j(x) = \mathbb{1}\left[\, y(x) = c_j \,\right]$



Appariement hongrois, puis perte

Exemple : 6 requêtes, 3 classes présentes dans l'image

	route	voiture	personne
requête 1			apparié
requête 2	apparié		
requête 3			
requête 4			
requête 5		apparié	
requête 6			

Requêtes 3, 4 et 6 non appariées : leur cible devient « aucun objet ». Chaque masque réel a exactement une requête.

1. Coût de chaque association, puis appariement optimal

$$\mathcal{C}(j,q) = -2\,p_q(c_j) + 5\,\mathrm{BCE}(m_q, g_j) + 5\,\mathrm{Dice}(m_q, g_j)$$

$$\hat{\sigma} = \arg\min_{\sigma} \sum_{j=1}^{K} \mathcal{C}\big(j, \sigma(j)\big)$$

2. Perte sur cet appariement, à chaque étape l

$$\mathcal{L} = \sum_{l} \Big[2\,\mathcal{L}_{\mathrm{cls}}^{(l)} + 5\,\mathcal{L}_{\mathrm{BCE}}^{(l)} + 5\,\mathcal{L}_{\mathrm{Dice}}^{(l)} \Big]$$

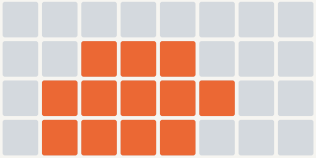
- « aucun objet » pondéré 0,1 dans l'entropie croisée
- supervision profonde : 10 étapes (Mask2Former), 4 (EoMT)

L'attention masquée

$$X_l = \mathrm{softmax}\left(\mathcal{M}_{l-1} + \frac{Q_l K_l^\top}{\sqrt{d}}\right) V_l + X_{l-1}$$

$$\mathcal{M}_{l-1}(q,x) = \begin{cases} 0 & \text{si } \sigma\left(m_q^{(l-1)}(x)\right) \geq 0{,}5 \\ -\infty & \text{sinon} \end{cases}$$

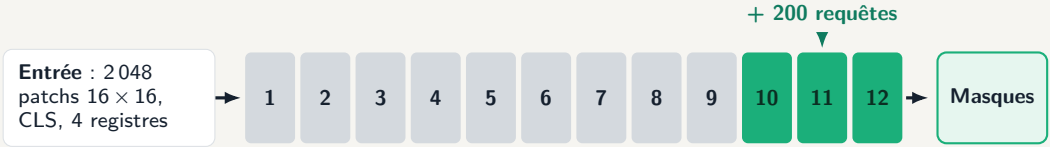
Chaque requête ne regarde que les pixels de son masque prédit à la couche précédente : le masque s'affine de couche en couche, l'apprentissage converge plus vite.



Orange : masque courant de la requête, attention autorisée. **Gris** : $-\infty$, ignoré.

Swin-Tiny : cartes à 1/4, 1/8, 1/16, 1/32 → **Pixel decoder** : 6 couches d'attention déformable → **Décodeur** : 9 couches, 100 requêtes, échelles 1/32, 1/16, 1/8 → **Masques** : requête · pixel, à 1/4

Un ViT encodeur seul, sans décodeur



Gris : blocs ViT-B pré-entraînés DINOv2 · vert : requêtes et patchs ensemble dans l'auto-attention

Module de masques

$$\mathbf{p}_n = \mathrm{softmax}\big(W\mathbf{q}_n\big)$$

$$m_n = \big\langle \mathrm{MLP}_3(\mathbf{q}_n),\ \mathcal{U}(T) \big\rangle$$

$\mathcal{U}$: 2 convolutions transposées (1/16 vers 1/4)

Pourquoi plus rapide

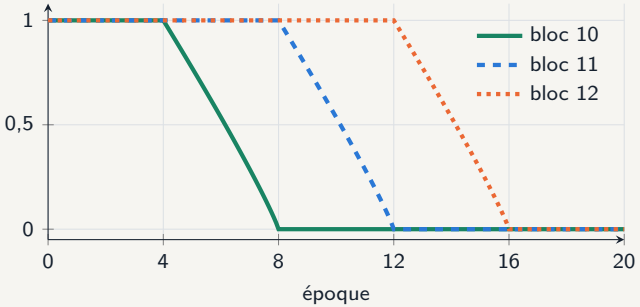
- aucun décodeur
- uniquement des opérations de ViT, rapides en bf16
- aucun masque à calculer à l'inférence

Adapté au 512×1024

Modèle COCO en 640×640 (grille 40×40) : position embeddings interpolées en bicubique vers 32×64.

Mask annealing : retirer les masques

Probabilité de garder l'attention masquée, par bloc (entraînement réel, 20 époques)



$$P_b(t) = \left(1 - \frac{t - t_b^s}{t_b^e - t_b^s}\right)^{0,9}$$

pour $t_b^s \leq t < t_b^e$; retrait entre les pas 2 976 et 11 904 sur 14 880.

Validation toujours sans masque, comme en production : la mIoU monte pendant le retrait (0,835 → 0,842).

Recette d'entraînement

Élément	ResNet50-UNet	Mask2Former	EoMT
Pré-entraînement	encodeur ImageNet	modèle entier ADE20K	modèle entier COCO (ViT DINOv2)
Optimiseur	Adam	AdamW, décroissance 0,05	AdamW, décroissance 0,05
Taux	10^{-4} décodeur, 10^{-5} encodeur, constant	10^{-4}, backbone 10^{-5} ; warm-up de 250 pas, puis décroissance polynomiale	$10^{-4} \times 0{,}8^{\,11-l}$ pour la couche l ; warm-up des têtes puis du ViT
Perte	CE + Dice pondérées	appariement + CE, BCE, Dice	idem, une perte par bloc
Lot, précision	4, fp32	2×2 accumulés, bf16, gradient $\leq 0{,}01$	2×2 accumulés, bf16, gradient $\leq 0{,}01$
Spécifique	poids de classe de 0,5 à 3	tête 150 $\rightarrow$ 9 classes	embeddings $40 \times 40 \rightarrow 32 \times 64$, mask annealing

RÉSULTATS

EoMT : +0,027 de mIoU

ResNet50-UNet · baseline

0,816

mIoU validation
erreur restante 0,184

Mask2Former

0,842

mIoU validation
+0,026 vs baseline
erreur restante −14 %

EoMT · nouvelle méthode

0,843

mIoU validation
+0,027 vs baseline
erreur restante −15 %

Lecture : à ce niveau, chaque point coûte cher. La baseline laisse 0,184 d'erreur, EoMT 0,157 : **15 % de l'erreur restante en moins**. Sur les classes difficiles, davantage : fond −29 %, personnes −15 %.

500 images de validation Cityscapes, 9 classes, 512×1024 · les trois modèles ré-évalués par le même script

RÉSULTATS · PAR CLASSE

Gains sur les classes difficiles

IoU par classe	Baseline	Mask2Former	EoMT	Gain EoMT
fond	0,755	0,813	0,827	+0,071
personne	0,728	0,769	0,770	+0,042
bâtiment	0,860	0,892	0,893	+0,032
panneau	0,653	0,698	0,680	+0,027
vélo	0,691	0,713	0,718	+0,027
route	0,926	0,946	0,953	+0,027
voiture	0,914	0,930	0,934	+0,020
végétation	0,895	0,903	0,902	+0,007
ciel	0,918	0,912	0,907	−0,011

Lignes surlignées : les classes rares ou hétérogènes, où la baseline était faible.

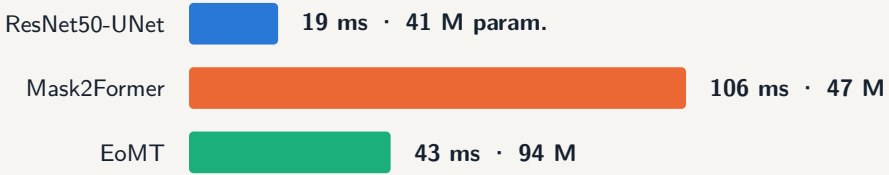
8 classes sur 9 progressent, toutes de façon significative (bootstrap).

Seule baisse : le ciel (−0,011), grande zone simple déjà bien traitée.

RÉSULTATS · VITESSE

Même précision, 2,4 × plus rapide

Temps d'inférence d'une image 512 × 1024 · GPU portable RTX 2000 Ada, bf16



Le compromis

EoMT : même précision que Mask2Former, 2,4 × plus rapide. La baseline reste 2 × plus rapide qu'EoMT, pour 0,027 de mIoU en moins.

Sur CPU (cloud)

Sans GPU ni bf16, l'avantage d'EoMT disparaît : environ 1,6 s, 3,2 s et 4,1 s par image sur 2 CPU.

RÉSULTATS · FIABILITÉ

Des écarts réels et reproductibles

Bootstrap apparié, 2 000 tirages

Écart de mIoU	Δ	IC 95 %	Verdict
Mask2Former − base	+0,026	[0,021 ; 0,031]	réel
EoMT − base	+0,027	[0,022 ; 0,032]	réel
EoMT − Mask2Former	+0,001	[−0,003 ; 0,005]	égalité

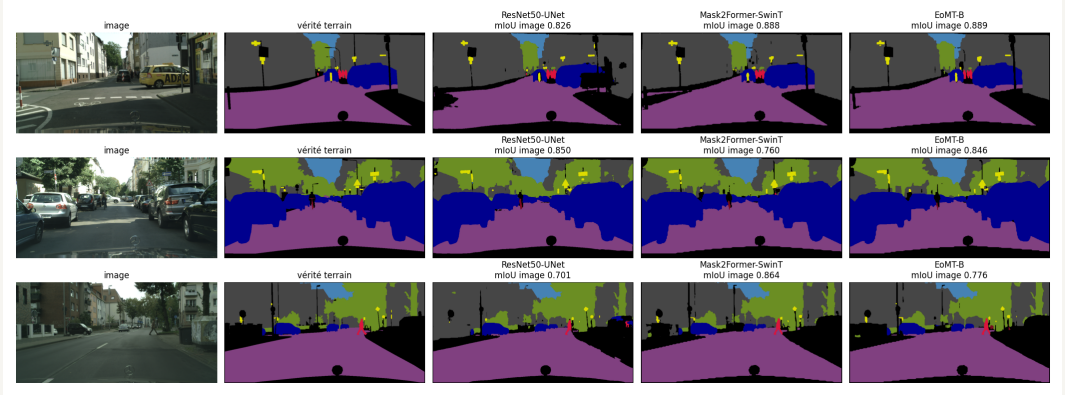
On ré-échantillonne 2 000 fois les 500 images de validation (tirage avec remise) : l'écart avec la baseline reste toujours positif.

Ré-entraînement complet

mIoU	Notebook	Référence
ResNet50-UNet	0,8152	0,8157
Mask2Former	0,8402	0,8419
EoMT	0,8423	0,8426

Les trois modèles ré-entraînés de zéro dans le notebook : écarts inférieurs à 0,002.

Contours plus nets, moins de bruit



Colonnes : image, vérité terrain, ResNet50-UNet, Mask2Former, EoMT. La mIoU d'une seule image varie beaucoup : les conclusions viennent des 500 images.

Limites de la méthode, pistes d'amélioration

Limites d'EoMT

- **Dépend du pré-entraînement** : sans ViT pré-entraîné (DINOv2), aucun décodeur ne compense
- **Lourd** : 94 M paramètres (baseline 41 M)
- **Coût quadratique** : en pleine résolution, $4\times$ plus de patchs, $\approx 16\times$ plus d'attention
- **Rapide seulement sur GPU** : sur CPU, 4,1 s contre 1,6 s (baseline)
- **Petits objets** : panneaux 0,680, vélos 0,718 ; panneaux sous Mask2Former (COCO ? hypothèse)

Pistes d'amélioration

- **Résolution native** : un Mask2Former Cityscapes passe de 0,708 à 0,742 (test sur 10 images)
- **Augmentation** de l'article : échelles, recadrages, retournements
- **Plus long** : 20 époques seulement ici
- **ViT-L** au lieu de ViT-B (GPU plus grand)
- **Sans ré-entraîner** : augmentation au test (retournement, multi-échelle)

Limites de l'étude et suites

Limites

- La validation sert à choisir l'époque et à mesurer : scores un peu optimistes, pour les trois modèles
- Pas d'augmentation de données : sur-apprentissage modéré (écart train − validation de $+0{,}06$ à $+0{,}08$)
- Points de départ pré-entraînés différents d'un modèle à l'autre
- Sans GPU, EoMT perd son avantage de vitesse

Suites proposées

- Plusieurs entraînements par modèle pour mesurer la variance
- Servir EoMT par l'API du projet, sur une machine avec GPU
- Quantification ou distillation pour un usage sur CPU
- Évaluer sur le jeu de test officiel via le serveur Cityscapes

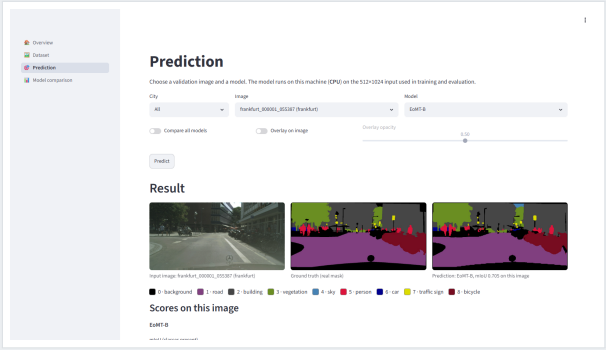
DÉMONSTRATION

Le dashboard, déployé sur le cloud

huggingface.co/spaces/DaryaEL/cityvision-poc

1. **Dataset** : comptages, exemples par ville et par classe, égalisation, CLAHE, flou
2. **Prediction** : choix d'une image et d'un modèle, masque prédit et scores
3. **Model comparison** : précision, vitesse, bootstrap, courbes

Accessibilité (WCAG) : textes alternatifs, tableau sous chaque graphique, couleurs lisibles par les daltoniens, commandes au clavier.



CONCLUSION

EoMT bat la baseline et égale Mask2Former, en plus rapide

+0,027
de mIoU sur la baseline, significatif

2,4 ×
plus rapide que Mask2Former sur GPU

−15 %
d'erreur restante (1 − mIoU)

Merci. Questions ?